Gradum

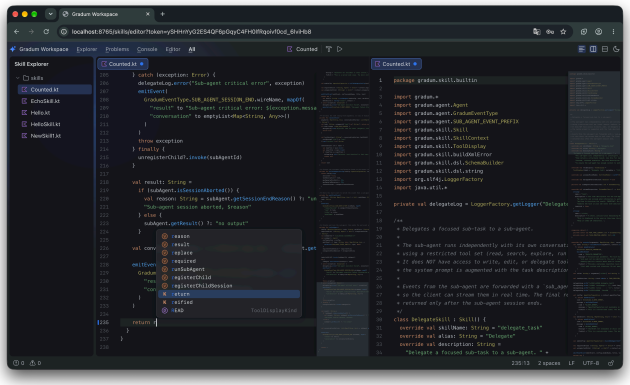
English | 简体中文

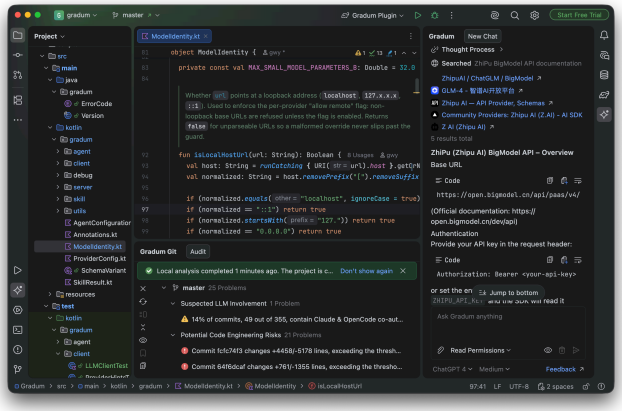


Gradum Runtime 是高自由度、轻量化来扩展 AI 智能体的本地运行时。它完全运行在你自己的机器上，为每个接入的客户端提供同套核心能力： 自选模型（本地 Ollama 或 LM Studio，也可以用你自己的 API Key 接入 BigModel、DeepSeek、MiniMax 或任何 OpenAI 兼容端点）；智能热插拔的技能，覆盖文件读写、项目结构探索和 Shell 命令执行；以及在破坏性操作执行前进行拦截的安全层。编码技能只是它自带的技能集，而不是它的边界，你可基于它进行任何改造。

你的代码、对话和 API 密钥永远不会离开你的机器。你可通过 IntelliJ IDEA 插件、内置浏览器技能编辑器或任何支持 ACP 的 IDE 客户端接入它。 运行时使用高效性能的 Kotlin 编写，同时也可作为构建你自己的智能体和工具的核心基础。

截图





运行时主要提供什么

- **多模型支持**：开箱即用地支持本地 Ollama（Cloud）、LM Studio、vLLM 和 LocalAI，以及智谱 BigModel、DeepSeek、MiniMax 等任何 OpenAI 兼容端点；
- **技能系统**：每个能力都以可热插拔的技能交付：内置编码技能、MCP 工具和外部技能目录；替换、扩展或新增技能都不需要重启
- **命令安全过滤器**：每条 Shell 命令在执行前都会被分类（阻止、需要批准或放行），破坏性操作需要你的批准，提权操作直接被阻止；列表可在 `settings.json` 中覆盖
- **会话连续性**：对话历史在重启后依然保留，并从上次中断的地方继续；每条记录在加载时都会校验，损坏或被篡改的条目在进入模型上下文之前就会被丢弃
- **流式事件**：NDJSON 事件流将思考过程、工具调用和回答实时推送给客户端
- **ACP 支持**：基于 Stdio 协议实现 Agent Client Protocol，权限请求桥接到客户端；任何支持 ACP 的 IDE 都能直接接入
- **HTTP API**：本地端点，为插件和其他本地集成提供统一接口

- **顺序编辑**：查找替换式编辑逐个应用，每个结果都有反馈
- **后台进程**：GUI 和长期运行的服务可以在不阻塞会话的情况下脱离
- **技能编辑器**：内置基于浏览器的智能 Kotlin IDE，位于 `GET /skills/editor`，自带语法高亮、问题面板和缩略图。点击 Run 会将源码部署到 `~/.gradum/skills`，热重载监视器会立即接管

快速开始

从 Releases 页面下载对应平台的压缩包，每个发行版都是自带精简后运行时，你无需安装繁重的 JDK。

- `gradum-darwin-arm64.zip` 和 `gradum-darwin-x64.zip`（macOS 双击会打开终端窗口，用于实时滚动服务端日志）
- `gradum-linux-x64.zip` 和 `gradum-linux-arm64.zip`
- `gradum-windows-x64.zip`

请先运行配置向导：

```
java -jar gradum@1.0.2.jar setup    # 首次运行：选择提供商、密钥、Base URL
```

向导会把你的选择写入 `~/.gradum/settings.json`（同时生成 `settings.schema.json` 供编辑器自动补全）。 启动服务端、打开编辑器、连接 IDE，读取的都是该文件，旧的 `--host`、`--port`、`--api-key` 参数已被移除。

Gradum Runtime 提供 3 个入口：

命令	作用
（无）	启动 HTTP 服务端，NDJSON `/events` 流式 API
`setup`	启动交互式终端提供商配置向导
`acp`	面向 IDE 智能体客户端的 ACP Stdio 服务端

```
java -jar gradum@1.0.2.jar --help   # 用法、配置概览、端点列表
java -jar gradum@1.0.2.jar acp      # 基于标准输入/输出的 Agent Client Protocol
```

服务端启动后，在浏览器中打开 `GET /skills/editor` 即可使用技能编辑器。

安装 IntelliJ IDEA 插件

插件不提供预构建压缩包，需要从源码构建（需要使用 JDK 25）：

```
./gradlew :plugin:buildPlugin
```

通过 `Settings > Plugins > Install Plugin from Disk` 安装 `plugin/build/distributions/gradum-*.zip`，然后将插件指向一个正在运行的 Gradum Runtime。

ACP 客户端

`gradum acp` 基于 stdin/stdout 实现 Agent Client Protocol。ACP agent card 位于 `registry/gradum/agent.json`（版本号、各平台下载摘要、`args: ["acp"]`），支持注册表的客户端可以直接安装并启动 Gradum。当还没有配置任何提供商时，initialize 响应会声明 `terminal` 认证方式，在终端中运行 `gradum setup`。

配置文件

`~/.gradum/settings.json` 是唯一的配置来源，旁边附带 JSON Schema 文件，可用于编辑器自动补全：

```
{
  "server": {
    "host": "localhost",
    "allowRemote": false,
    "port": 8765,
    "autoDetectPort": false,
    "apiKeyFile": null
  },
  "llm": {
    "baseUrl": "http://localhost:11434",
    "model": "",
    "think": false
  },
  "commandFilter": {
    "blockedExecutables": ["reboot", "shutdown", "..."],
    "protectedPaths": { "...": "..." }
  },
  "mcpServers": [
    { "name": "idea", "command": "...", "workingDir": "...", "env": {} }
  ],
  "plugins": {},
  "ollama.baseUrl": "http://localhost:11434",
  "ollama.apiKey": "",
  "zhipu.apiKey": "",
  "deepseek.apiKey": "",
  "minimax.apiKey": "",
  "lmstudio.baseUrl": ""
}
```

未知的键会在启动时报告，而不是被静默忽略；整个文件会以紧凑的问题树形式校验（`Invalid gradum.settings (fix ~/.gradum/settings.json)`）。想让局域网内其他设备访问编辑器，需要同时设置 `"host": "0.0.0.0"` 和 `"allowRemote": true`；非环回地址但没有这个显式开启的配置会拒绝启动，且每个请求仍需携带本机的 Bearer Token。服务端日志输出到 stdout，同时滚动写入 `~/.gradum/logs/`（ACP 传输额外写入 `~/.gradum/logs/acp.log`）。

LLM 后端配置

`search_web` 技能使用 Tavily Search API；设置 `TAVILY_API_KEY` 即可启用（免费额度为每月 1,000 次调用）。其余技能均不需要密钥。

提供商的 Base URL 和 API Key 存放在 `~/.gradum/settings.json`；推荐的写入方式是 `java -jar gradum@1.0.2.jar setup` 或插件的设置页面。环境变量可以作为云端密钥的兜底：

```
export ZHIPU_API_KEY="your-key-here"       # 智谱 BigModel（GLM）
export DEEPSEEK_API_KEY="your-key-here"    # DeepSeek
export MiniMax_API_KEY="your-key-here"     # MiniMax（注意大小写）
export TAVILY_API_KEY="your-key-here"      # 网络搜索技能
```

本地提供商（Ollama、LM Studio）无需密钥；它们的 Base URL 分别读取 `ollama.baseUrl` 和 `lmstudio.baseUrl`。

IntelliJ IDEA 插件

IDEA 插件把 Gradum Runtime 带进你已经在用的工作区。交给它任务：它会读取任务涉及的文件，在改动落地前呈现给你审查，并在执行任何命令前先征求你的意见。

- **扎根于你的机器**：连接本地 Ollama 或 LM Studio，或者你自己的云端模型服务；数据留在你的机器上，不经过任何第三方
- **技能即代码**：智能体的工具就是 Kotlin 技能；在内置浏览器编辑器中修改它们，下轮对话即生效
- **命令安全**：每条 Shell 命令在执行前都会被分类；破坏性操作需要你的批准，提权操作直接被阻止
- **会话恢复**：对话历史在重启后依然保留；记录在加载时会被校验，损坏或被篡改的条目永远不会进入模型上下文
- **Git 风险统计分析**：底部工具窗口审查你的本地提交历史，按主题和严重程度呈现风险发现，并计算项目质量档位

其他能力包括流式响应、思考预览、工具调用指示器、富文本 Markdown 渲染、深浅色主题、文件和图片附件、以及发送当前正在编辑的文件等。插件使用 Kotlin 以及优秀且先进的 Jewel 框架构建，当前插件版本为 1.0.4。

安全性

命令安全过滤器在任何东西运行之前对 Shell 命令进行分类：

- **阻止**：破坏性系统工具（`mkfs*`、`mkswap`、`fdisk`、`sfdisk`、`parted`、`gdisk`、`shutdown`、`reboot`、`poweroff`）
- **需要批准**：有风险但合法的命令（`dd`、破坏性的 `rm`、权限变更等）；智能体会在执行前询问
- 提权命令（`sudo`、`su`、`doas`、`pkexec`）通过结构方式识别，过滤器能看穿这类包装
- 受保护路径（`/etc`、`/usr`、`~/.ssh` 等）单独防护，但 `/tmp` 始终放行

被阻止的可执行文件和受保护路径可以通过 `settings.json` 的 `commandFilter` 部分覆盖。

本地 HTTP API 本身也有纵深防护：`Host` 白名单拒绝 DNS 重绑定尝试，除 `/health` 外的每个路由都需要存放在 `~/.gradum/server.token` 中的 Bearer Token（0600 权限；插件读取同一文件），技能编辑器用 `?token=` 兑换 `HttpOnly` Cookie， 并且当 `server.host` 不是环回地址时，服务端拒绝启动。

从源码构建

服务端需要 JDK 21，IntelliJ 插件需要 JDK 25。仓库不固定 JDK 路径，Gradle 将会使用你 `JAVA_HOME` 中的 JDK 或 IDE 中选择的 Gradle JVM：

```
./gradlew buildFatJar            # 跨平台 fat jar，输出到 build/libs/
./gradlew :plugin:buildPlugin    # IDEA 插件 zip，输出到 plugin/build/distributions/
```

构建不是单 JDK、单命令就能完成的事（双 JDK 要求、IDE 复制的 jar 和 fat jar 打包各有已知的限制）。如果构建失败，请先查阅 CONTRIBUTING.md 中的完整检查清单。

文档

- 架构：系统设计、数据流、模块、ACP 传输、打包的详细阐述
- 插件功能：详细功能索引：聊天、Markdown 管线、Git 分析、国际化的详细阐述
- 编码规范：项目的 Kotlin 编码格式约定指导
- 技能开发：创建用户自定义技能指南
- 基础设施陷阱：Jewel、Compose 和 Classloader 集成的限制记录
- 日志指南：日志命名与级别约定规则

- 版本规则：版本号提升规则与发行命名规则
- 更新日志 · 隐私政策 · 参与贡献

参与贡献

1. Fork 本仓库
2. 创建功能分支：`feature/@your-name-add-thing`
3. 使用描述清晰的提交信息
4. 发起 Pull Request 即可

许可证

MIT License，见 LICENSE

Gradum 版本

服务端 1.0.2，插件 1.0.4